APPENDIX E

Glossary

This glossary defines the main terms of the book: the vocabulary of type theory, logic and category theory, and the Rust and Lean terms that the chapters connect to it. Each definition is short and says what the term means in this book; the chapter at the end of the entry is the one that introduces the term or explains it best, and is the place to go for the full story. Closely related terms share an entry, listed under the first of them. Terms that begin with a Greek letter are listed under the letter's name: α under A, β under B, η under E, λ under L, Π under P and Σ under S. Appendix C lists the symbols.

In this appendix you will learn:

- what the central terms of type theory, logic and category theory mean, as the book uses them;
- the Rust and Lean features that the chapters connect to each term;
- which words the book uses in more than one sense;
- which chapter to read for the details of each term.

E.1 A

Abstract syntax. The tree structure of a program, which records which operation is applied to which subexpressions and forgets the characters the programmer typed (the concrete syntax). Type checkers and evaluators work on it. (Chapter 3)

Ad-hoc polymorphism. One name with different code at different types, such as `+`, `==` or `to_string`. Type classes, which Rust calls traits, make it as principled as the parametric kind. (Chapter 14)

Adjunction. A pair of functors L and R with a bijection $\mathrm{Hom}(L\,a, c) \cong \mathrm{Hom}(a, R\,c)$, natural in a and c, written $L \dashv R$. Currying is the first example, and every adjunction gives rise to a monad. (Chapter 19)

Admissible rule, derivable rule. A derivable rule abbreviates a fixed combination of the primitive rules, like a macro. An admissible rule only promises that its conclusion is derivable whenever its premises are; proving that usually takes an induction on derivations. (Chapter 6)

Affine λ-calculus. The programming language whose type system is affine logic, written λ^{aff}: typing rules with context splitting, in which weakening is allowed and contraction absent. It is the formal account of Rust's moves and of error E0382. (Chapter 22)

Affine logic. The logic that keeps weakening and drops contraction: every assumption is used at most once, and resources may be thrown away but not copied. It is the discipline of Rust's owned values that are not `Copy`. (Chapter 21)

Algebraic data types. Types built from products, sums, the unit type, the empty type and function types, such as Rust's structs and enums. If you count values, products multiply, sums add and function types are powers, hence the name. (Chapter 11)

Aliasing XOR mutation. The principle behind Rust's references: at any moment, a place may have many readers or one writer, never both. (Chapter 26)

α-equivalence. Two terms are α-equivalent, $t \underset{\alpha}{\equiv} u$, when they differ only in the names of bound variables, renamed consistently and without capture. They denote the same program. (Chapter 7)

Associated type. A type that each implementation of a trait must define, such as `Iterator::Item`, and that the implementing type therefore determines. Haskell's functional dependencies and Lean's `outParam` express the same idea. (Chapter 14)

Axiom. An inference rule with no premises, whose conclusion holds outright. In Lean the word also names the few assumptions of the logic itself, such as `propext` and `Classical.choice`; `#print axioms` lists the ones a theorem depends on. (Chapter 1)

E.2 B

β-reduction. The computation rule of the λ-calculus: a redex $(\lambda x.\, t)\, s$ reduces to $t[x := s]$. Under Curry–Howard it is the removal of a detour from a proof. (Chapter 7)

Bidirectional type checking. A type checking method that splits the typing judgment in two: synthesis, $\Gamma \vdash t \Rightarrow \tau$, where the type comes out of the term, and checking, $\Gamma \vdash t \Leftarrow \tau$, where an expected type flows in. Annotations are needed only where neither direction knows the type. (Chapter 8)

Big-step and small-step semantics. Two styles of operational semantics. A big-step semantics relates an expression directly to its final value, as in $\rho \vdash e \Downarrow v$ (Chapter 3); a small-step semantics describes one step of computation at a time, $t \to t'$, which is the style of the type safety proof (Chapter 9).

Borrow. A temporary transfer of permission from an owner. `&x` lends a read permission that the owner keeps too; `&mut x` lends the owner's read-write permission, and the owner is unusable until the borrow's region ends. (Chapter 26)

Borrow checker. The part of rustc that enforces Rust's rules for moves, borrows and lifetimes. It is not a type system in the textbook sense but a set of dataflow analyses over MIR. (Chapter 29)

Brouwer–Heyting–Kolmogorov interpretation. BHK. The constructive reading of the connectives as kinds of proof: a proof of $A \vee B$ is a proof of one side together with the information which, and a proof of $A \to B$ is a method that turns proofs of A into proofs of B. (Chapter 5)

E.3 C

Calculus of Constructions. The corner λC of the λ-cube, due to Coquand and Huet (1988), in which terms and types live in one language and any of them may depend on any other. Lean's type theory extends it with inductive types, universes and quotient types. (Chapter 30)

Cartesian closed category. CCC. A category with a terminal object, binary products and exponentials. The simply typed λ-calculus can be interpreted in any CCC, which completes the Curry–Howard–Lambek triangle. (Chapter 19)

Category. A collection of objects and of morphisms (arrows) $f : a \to b$ between them, with an identity morphism on every object and an associative composition $g \circ f$ ("first f, then g") for which the identities are units. Equations between composites are drawn as commutative diagrams. (Chapter 16)

Church encoding. A representation of data by what you can do with it, using only λ-terms: booleans as functions that choose, numerals as functions that iterate, pairs as functions that hand over two components. (Chapter 7)

Classical logic. The logic of truth tables, obtained from intuitionistic natural deduction by adding excluded middle, $A \vee \neg A$, or an equivalent rule such as *reductio ad absurdum*. It is not wrong, but its proofs of $A \vee B$ need not say which side holds. In Lean, excluded middle is `Classical.em`, which follows from the axiom `Classical.choice`. (Chapter 5)

`Clone`. The Rust trait of types that can be duplicated explicitly, by a call to `clone` that may cost something and may have effects. In the affine λ-calculus it is a term of type $\tau \multimap \tau \otimes \tau$, not a structural rule. Every `Copy` type is `Clone`. (Chapter 23)

Closure. A function value together with the values of its free variables, captured from the environment in which it was created. The captures of a Rust closure are the free variables of its body. (Chapter 8)

Coercion. An implicit conversion that the compiler inserts where the expected type is known, such as an unsizing coercion to `Box<dyn Trait>` or a deref coercion. Unlike Rust's subtyping between lifetimes, these coercions do real work at run time. (Chapter 27)

Coherence. The property of a type-class system that all derivations of a constraint elaborate to programs with the same behavior, so that the meaning of a program does not depend on which instance the compiler found. Rust ensures it by forbidding overlapping impls and by the orphan rule: an impl must live in the crate of the trait or in the crate of the type (E0117). (Chapter 14)

Confluence. The property that however two reductions of a term diverge, they can be brought back together. The Church–Rosser theorem says that the λ-calculus is confluent, so a term has at most one normal form. (Chapter 7)

Consistency. A logic is consistent when $\bot$ has no proof. Lean is consistent as a logic thanks to totality; Rust, read as a logic, is not, because `loop {}` and `panic!()` inhabit every type. (Chapter 10)

Const generic. A value-level parameter of a type or function, such as the `N` of `[T; N]`. It is Rust's first step toward dependent types; stable Rust allows no arithmetic on such parameters. (Chapter 30)

Constraint, entailment. A constraint is a statement about types, such as $\mathit{Monoid}\ \alpha$, which a qualified type $\pi \Rightarrow \tau$ requires; Rust's trait bounds are constraints. Entailment, $P \Vdash \pi$, is the judgment that the constraints P imply π, and each instance declaration is one of its rules. (Chapter 14)

Context. The assumptions of a judgment, usually written Γ: in type theory, the variables in scope with their types; in logic, the propositions assumed. (Chapter 1)

Context splitting. Dividing the assumptions of a rule's conclusion between its premises, $\Gamma = \Gamma_1 \uplus \Gamma_2$, so that each assumption goes to exactly one of them. It is how substructural systems refuse contraction, and in the affine λ-calculus it is the move checker. (Chapter 22)

Continuation-passing style. CPS. A style in which a computation, instead of returning its value, passes it to a continuation that says what to do with it. The type $\forall\beta.\,(\alpha \to \beta) \to \beta$ is isomorphic to α. (Chapter 18)

Contraction. The structural rule that merges two copies of an assumption into one, so that an assumption may be used more than once. For programs it is duplication, which Rust refuses for values that are not `Copy`. (Chapter 6)

Coproduct. The dual of a product: an object with two injections through which every pair of morphisms out of the components factors uniquely. In the category of types it is the sum type. (Chapter 16)

`Copy`. The Rust trait of types whose values are duplicated implicitly, by a bitwise copy, so that using them does not move them. It behaves like the exponential ! of linear logic: a type may be `Copy` only if all its parts are, and never if it has a destructor. (Chapter 23)

Covariant, contravariant. A type constructor F is covariant when $\sigma <: \tau$ implies $F\,\sigma <: F\,\tau$, as `&'a T` is in `'a`, and contravariant when it implies $F\,\tau <: F\,\sigma$, as `fn(T)` is in `T`. Categorically, they are functors that keep or reverse the direction of arrows. (Chapters 17 and 27)

Curry–Howard correspondence. The fact that the typing rules of a typed λ-calculus and the rules of a logic are the same rules: propositions are types, proofs are programs, and simplifying a proof is running a program. (Chapter 10)

Currying. Turning a function of two arguments into a function that returns a function, $(A \times B \to C) \cong (A \to B \to C)$. In category theory it is the first example of an adjunction. (Chapter 7)

Cut. The rule that proves a lemma and then uses it. Cut elimination, Gentzen's *Hauptsatz*, says that every derivation in the sequent calculus can be transformed into one without cuts. (Chapter 4)

E.4 D

Dataflow analysis. A static analysis that computes facts at every point of a control-flow graph by letting them flow along the edges until nothing changes, which is a fixpoint. rustc's move analysis, liveness and loan tracking are dataflow analyses. (Chapter 29)

de Bruijn index. A nameless representation of a bound variable: the number of λs between the occurrence and its binder. With de Bruijn indices, α-equivalent terms are identical. (Chapter 7)

`Decidable`. The Lean class of propositions for which an algorithm returns either a proof or a refutation: a decision procedure. It is what lets `if` test a proposition and what the tactic `decide` evaluates. (Chapter 14)

Deep embedding, shallow embedding. Two ways to study a logic in Lean. A shallow embedding uses Lean's own propositions and connectives, which is convenient for proving things in the logic but

not about it. A deep embedding defines the syntax and the rules as ordinary inductive types, so that formulas and derivations are values that theorems can talk about. (Chapter 4)

Definitional equality, propositional equality. Definitional equality is what the type checker sees by itself, by unfolding definitions, β-reduction and pattern matching on constructors; `rfl` proves exactly these equations. Propositional equality is the type `a = b` of everything that can be proved equal, possibly by induction; every definitional equality is propositional, not the other way around. (Chapter 30)

Denotational semantics. A semantics that maps the syntax of a language to mathematical objects, such as Lean values or the morphisms of a cartesian closed category. (Chapter 8)

Dependent type. A type that mentions a value, such as the type of arrays of length `n` or the type of proofs that `i < n`. Lean has them in full; Rust approximates them with const generics. (Chapter 30)

Derivation. A tree of rule instances whose root is the judgment to be established and whose leaves are axioms. A judgment holds exactly when it has a derivation, and type checking is the search for one. (Chapter 1)

Detour. An introduction rule immediately followed by an elimination of the same connective. Local soundness says that every detour can be removed; under Curry–Howard a detour is a redex. (Chapter 4)

Diagonal. The morphism $\langle \mathrm{id}, \mathrm{id} \rangle : a \to a \times a$ that pairs a value with itself: the categorical name for using a value twice, and so for contraction. Rust provides it freely only for `Copy` types. (Chapter 16)

Dictionary passing. The translation of a program with type classes into one without them, in which every constraint becomes an extra argument, a record of the class's operations. Rust's generics usually remove the dictionaries by monomorphization, and `dyn Trait` keeps them as vtables. (Chapter 14)

Disjunction property, existence property. If $A \vee B$ is provable intuitionistically with no assumptions, then A is provable or B is; if $\exists x.\, A(x)$ is provable, then $A(t)$ is provable for some term t. Constructive proofs carry the information which side holds, and a witness. (Chapter 5)

Double-negation translation. An embedding of classical logic into intuitionistic logic that inserts double negations. For propositional logic it is Glivenko's theorem: A is a classical tautology if and only if $\neg\neg A$ is intuitionistically derivable. (Chapter 5)

`Drop`. The Rust trait for destructors: code that runs when a value is discarded, to release what it holds. It turns weakening into an action with an effect, which is why a type with a `Drop` impl cannot be `Copy`. (Chapter 24)

Dyn compatibility. The property of a trait that allows `dyn Trait`, called object safety in older documentation. Its rules follow from the typing of existential packages and from the vtable representation; violations are E0038. (Chapter 15)

E.5 E

Effect. Anything a computation does besides returning a value: failing, producing several answers, reading or updating state, waiting. Functions with effects do not compose like plain functions; monads make them compose again. (Chapter 20)

Elaboration. Translating what the programmer wrote into a fully explicit term. Lean's elaborator fills in implicit arguments, instances and coercions by unification (Chapter 13), and a type checker for type classes elaborates constraints into dictionary arguments (Chapter 14).

Elimination restriction. Lean's rule that a proof of a proposition can be taken apart only to build another proof. That is why the witness of an `∃` cannot be extracted as data. (Chapter 10)

Empty type. A type with no values, written $\mathbf{0}$: Rust's `enum Void {}` and `Infallible`, Lean's `Empty`. It is "false" under Curry–Howard and an initial object in the category of types, and a `match` on it needs no arms. (Chapter 11)

Escape check. The rule that a value whose type mentions a region may not leave that region, written $\rho \in \Delta$ in the region discipline of Chapter 26 and $\alpha \notin \mathrm{flv}(\sigma)$ in the borrow calculus of Chapter 28. Rust reports violations as E0597 or E0515. (Chapter 26)

η-rules. Equations that say a value is determined by what you can do with it: $\lambda x.\, f\, x$ is f, and the pair of the two projections of p is p. Logically they are local completeness. (Chapter 8)

Exchange. The structural rule that lets two neighboring assumptions swap places. Ordered logic drops it; Rust keeps it. (Chapter 6)

Existential type. A type $\exists \alpha.\, \tau$ that packs a hidden type with a value that uses it; clients may use the package only through τ. Abstract data types, `impl Trait` in return position (an opaque type) and `dyn Trait` are existentials. (Chapter 15)

Exponential. The function type $A \to B$, written B^A because it has $|B|^{|A|}$ values (Chapter 11). In a category, the object $b \Rightarrow c$ that comes with eval and curry and satisfies their universal property (Chapter 19).

Exponential modality. The connective $!A$ of linear logic, "of course A", which marks the formulas that may be duplicated and discarded. Its rules are weakening, contraction, dereliction (use a $!A$ once, as an A) and promotion, and Rust's `Copy` behaves like it. (Chapter 23)

E.6 F

Fixed-point combinator. A term such as Curry's Y that, applied to a function g, produces a fixed point of g. It gives the untyped λ-calculus recursion; Z is the variant that works under call-by-value. (Chapter 7)

`Fn`, `FnMut`, `FnOnce`. Rust's closure traits, ordered by how often a closure may be called: `FnOnce` at most once, an affine function; `FnMut` any number of times, one call at a time; `Fn` any number of times, even from several places at once. (Chapter 8)

Free theorem. An equation that every function of a given polymorphic type satisfies, read off the type by parametricity. For example, every function of type $\forall \alpha.\, \alpha \to \alpha$ is the identity. (Chapter 18)

Free variable, bound variable. In $\lambda x.\, x\, y$, the x is bound: it refers to the enclosing binder. The y is free: no binder in the term binds it, and its meaning must come from outside. (Chapter 7)

Functor. A map between categories, on objects and on morphisms, that preserves identities and composition (the functor laws). For types, it is a type constructor with a lawful `map`; a functor from a category to itself is an endofunctor. (Chapter 17)

E.7 G

Generic associated type. GAT. An associated type that takes parameters of its own, stable since Rust 1.65. It lets a trait say "the same container, with a different element type", a partial workaround for higher-kinded types. (Chapter 17)

Goal, infoview. The goal is what remains to be proved in a Lean proof. The infoview, the panel of a Lean editor that shows what Lean knows at the cursor, displays it after the turnstile `⊢`, below the hypotheses. (Chapters 1 and 2)

E.8 H

Harmony. The requirement that the elimination rules of a connective extract exactly what its introduction rules put in. Its two halves are local soundness (a detour can be removed, the source of β-reduction) and local completeness (a formula can be taken apart and rebuilt, the source of η). Prior's `tonk` shows what goes wrong without it. (Chapter 4)

Heyting algebra. A bounded lattice with an implication characterized by residuation: $c \leq (a \Rightarrow b)$ if and only if $c \sqcap a \leq b$. Heyting algebras are the algebraic semantics of intuitionistic logic, as Boolean algebras are of classical logic. Categorically, a Heyting algebra is a thin bicartesian closed category. (Chapter 5)

Higher-kinded types. Type parameters that range over type constructors such as `Option`, rather than over types. Rust lacks them, which is why it has no `Functor` or `Monad` trait. (Chapter 17)

Higher-ranked trait bound. HRTB. A bound written with `for<'a>`, which requires a trait to hold for every lifetime, for example of a callback that must accept references of any lifetime. It gives Rust a form of rank-2 polymorphism. (Chapter 12)

Hindley–Milner. The type system of ML, in which polymorphism is restricted to `let`-bound variables: generalization turns the type of a `let` definition into a type scheme $\forall \alpha_1 \ldots \alpha_n.\, \tau$, and each use takes an instance. Its types can be inferred without annotations, and Milner's Algorithm W finds principal types. Rust's local inference is a restricted cousin. (Chapter 13)

Hoare triple. A specification $\{P\}\ c\ \{Q\}$: from any state satisfying the precondition P, the command c runs without fault to a state satisfying the postcondition Q. (Chapter 32)

E.9 I

Impredicativity. A quantifier is impredicative when its range includes the type it is part of: in Lean, `∀ p : Prop, p → p` is itself a `Prop`. System F is impredicative; Lean's hierarchy of `Type`s is predicative, and impredicativity survives only in `Prop`. (Chapter 10)

Inductive definition, rule induction. An inductive definition defines a set as the smallest set closed under some rules, such as the grammar of expressions or a typing relation. Rule induction proves that every member has a property by showing that each rule preserves it. (Chapter 3)

Inductive family. An inductive type indexed by values, such as Lean's `≤` on natural numbers or the typing relation of a toy language. A function from values to types, such as a typestate `Conn : State → Type`, is called an indexed family. (Chapter 10)

Inductive predicate. An inductive type whose values are proofs, such as `HasType : Ctx → Expr → Ty → Prop`, in which each constructor is one rule. (Chapter 3)

Inference rule. A rule that says how a judgment follows from others: premises above a line, the conclusion below it, the name on the right. A rule is a schema whose instances replace its metavariables. (Chapter 1)

Instance. The book uses the word in two senses. A type is an instance of a type scheme when it is obtained by choosing types for the quantified variables (Chapter 13). An instance of a type class declares that a type belongs to the class and gives its operations; in Rust it is an `impl`, and instance resolution finds the instances a constraint needs, like Prolog solving Horn clauses (Chapter 14).

Interior mutability. Mutation through a shared reference, made safe by moving the aliasing check from compile time to run time, as `RefCell` does, or by never handing out references to the contents, as `Cell` does. (Chapter 26)

Intrinsic and extrinsic typing. In extrinsic typing, terms exist on their own and a separate typing relation says which are well typed. In intrinsic typing, the type of a term is part of its definition, as in a Lean type `Term Γ τ`, so that ill-typed terms cannot be constructed at all. (Chapter 8)

Introduction and elimination rules. An introduction rule says how to prove a formula with a given connective at the root, such as ∧I or →I; an elimination rule says how to use one, such as ∧E or →E. Under Curry–Howard they are the constructors and the destructors of a type. (Chapter 4)

Intuitionistic logic. The constructive logic of natural deduction without classical rules: to prove $A \vee B$ you must say which, and to prove $\exists x.\, A(x)$ you must produce an x. Under Curry–Howard it is the logic of programs. (Chapter 5)

Invariant. The book uses the word in two senses. A type constructor is invariant in a parameter when subtyping between arguments does not carry over in either direction, as `&mut T` is in `T` (Chapter 27). In a proof, an invariant is a property that every derivation or every reachable state has, used to show that something cannot happen (Chapter 21).

Inversion. Reasoning backwards from a derivable judgment to the premises that must have produced it. It works smoothly in a syntax-directed system, where the form of the expression determines the only rule that can conclude it; in Lean it is `cases` on the derivation. (Chapter 3)

Isomorphism. A morphism with an inverse; between types, a pair of functions that undo each other, written $A \cong B$. Isomorphic types are interchangeable but not equal. (Chapter 11)

E.10 J

Judgment. A statement about syntax that is established by a derivation, or not at all, such as $\Gamma \vdash e : \tau$. It belongs to the metalanguage: it is not a boolean expression of the program. (Chapter 3)

E.11 K

Kernel. The small, carefully engineered part of Lean that checks every proof term. Tactics and the elaborator search for proofs, and nothing is accepted that the kernel has not checked. (Chapter 1)

Kind. The type of a type expression: `Type` (some texts write $*$) for types that classify values, `Type → Type` for type constructors such as `Option`. (Chapter 17)

Kleisli arrow. A function $A \to M\,B$ whose effect is described by the monad M. Kleisli arrows are the morphisms of the Kleisli category of M, with `pure` as identities and Kleisli composition, and the monad laws are its category laws. (Chapter 20)

Kripke model. A semantics for intuitionistic logic: a preorder of worlds, read as states of knowledge, in which known atoms stay known. The forcing relation $w \Vdash A$, "at world w we know A", looks at every future world for implication and negation, and a two-world model refutes excluded middle. (Chapter 5)

E.12 L

λ-calculus. Church's calculus of functions, with only variables, abstractions $\lambda x.\, t$ and applications $t\ u$, which can nevertheless express every computation. The untyped version has terms that run forever; its typed versions are the core of type theory. (Chapter 7)

λ-cube. Barendregt's arrangement of eight typed λ-calculi by three kinds of dependency: terms on types (polymorphism), types on types (type operators) and types on terms (dependent types). (Chapter 30)

Leak. A value that is never dropped, through `mem::forget`, `Box::leak` or a cycle of `Rc` pointers. Leaking is safe in Rust: `mem::forget` was made safe before Rust 1.0, after the episode known as the Leakpocalypse, which is why Rust is affine and not linear, and why the soundness of `unsafe` code must not depend on a destructor running. (Chapter 24)

Leftovers. In algorithmic affine type checking, the part of the context that a subterm did not use, which is passed on to the next subterm. The judgment is $\Gamma \vdash e : \tau \dashv \Delta$. (Chapter 22)

Lifetime. In Rust, a name such as `'a` for a region of the program during which a reference may be used: a region variable. It is not how long a value lives. (Chapter 26)

Linear logic. Girard's logic (1987), in which every assumption is used exactly once. It drops both weakening and contraction, has two conjunctions and two disjunctions, and recovers reuse through the exponentials. (Chapter 21)

Liveness. A local is live at a program point if the value it holds there may be read later, along some path. Non-lexical lifetimes are built on liveness. (Chapter 29)

Loan. The record of a borrow expression in the borrow checker: which place was borrowed, how, and with which region. A loan is in scope at the points it can reach without leaving its region. (Chapter 29)

Logical relation. A family of predicates or relations on values, defined by recursion on types, that says what it means to behave like a value of a type. It is the main tool of semantic typing, and its fundamental theorem, from which type safety follows, says that every syntactically well-typed term is semantically well typed. (Chapter 32)

E.13 M

Mathlib. The library of mathematics maintained by Lean's community, with definitions, hundreds of thousands of theorems and many tactics. The book imports only the parts it uses. (Chapter 2)

Metavariable. The book uses the word in two senses. In a rule, a letter such as Γ, e or τ that stands for any context, term or type (Chapter 1). In Lean's elaborator, an unknown such as `?m.5` that unification must solve (Chapter 13).

MIR. rustc's mid-level intermediate representation: a control-flow graph of basic blocks, straight-line statements that act on places and end in a single terminator. The borrow checker runs on it. (Chapter 29)

Miri. An interpreter for MIR that checks every step for undefined behavior, including the aliasing rules of Stacked Borrows or Tree Borrows. It is a component of the nightly toolchain. (Chapter 32)

Monad. A type constructor M with $\mathit{pure} : A \to M\ A$ and $\mathit{bind} : M\ A \to (A \to M\ B) \to M\ B$ satisfying three laws (left identity, right identity and associativity): the structure that makes effectful functions compose. `Option`, `Result`, lists and state are monads, and monad transformers such as `StateT` combine several effects. (Chapter 20)

Monoid. A type with an associative operation and a neutral element, such as numbers with `+` and `0` or strings with concatenation and `""`. As a category, a monoid is a category with one object. (Chapter 14)

Monoidal category. A category with a product-like operation $\otimes$ and a unit object I, associative and unital up to isomorphism, without the diagonals and projections of a cartesian product. Symmetric monoidal closed categories are the models of linear logic. (Chapter 19)

Monomorphism, epimorphism. A monomorphism can be cancelled on the left, $f \circ g = f \circ h$ implies $g = h$; an epimorphism on the right. In the category of types they are exactly the injective and the surjective functions. (Chapter 16)

Monomorphization. Rust's implementation of generics: for every type at which a generic function is used, the compiler generates a separate copy of it. (Chapter 12)

Move. In Rust, using a value of a non-`Copy` type by value: ownership passes to the new place, and the old one may not be used again (E0382). In the affine λ-calculus, a move is a variable that goes to exactly one premise of a split. (Chapter 22)

Multiplicative and additive connectives. The two ways a rule of linear logic can treat the context: split it between the premises, as $\otimes$ does, or give all of it to each premise, as $\&$ does, so that only one of the premises can be used. (Chapter 21)

Multiplicity. How many times a variable may be used: once for an ordinary variable of an affine calculus, without bound for a `Copy` variable, not at all for a variable that a term did not receive. Affine types are about multiplicity; lifetimes are about extent, where a reference is valid. (Chapter 23)

`#[must_use]`. A Rust attribute that makes the compiler warn when a value is discarded on the spot. It is a lint, not a typing rule, and the closest Rust comes to relevant or linear types. (Chapter 24)

E.14 N

Natural deduction. Gentzen's proof system in which each connective has introduction and elimination rules, and a proof is a tree of judgments under assumptions. (Chapter 4)

Natural transformation. A family of morphisms $\eta_a : F\,a \to G\,a$, one for each object, such that the naturality square $G\,f \circ \eta_a = \eta_b \circ F\,f$ commutes. Polymorphic functions such as `Option::into_iter` and `reverse` are natural transformations. (Chapter 18)

Never type. Rust's type `!` of expressions that produce no value, such as `panic!(...)` and `loop {}` without a `break`. It has no values and coerces to every type, the type-theoretic version of "false". (Chapter 1)

Nominal typing, structural typing. A type is nominal when its identity is its declaration, so two structs with the same fields are different types, and structural when its identity is its structure, as for tuples, arrays and function-pointer types. (Chapter 3)

Non-lexical lifetimes. NLL. Rust's borrow checking since the 2018 edition, in which a region is a set of points of the control-flow graph, computed from liveness, rather than a lexical scope. (Chapter 29)

Normal form. A term that contains no redex, the "answer" of a computation. Not every untyped term has one (Ω does not), and by confluence no term has two. (Chapter 7)

Normalization. Reducing a derivation, or a term, until no detour remains. Prawitz's normalization theorem says that every derivation of natural deduction can be normalized; strong normalization, which the simply typed λ-calculus has and Rust does not, says that every reduction sequence ends. (Chapter 4)

E.15 O

Occurs check. The test that prevents unification from binding a type variable to a type that contains it, as in $\alpha \doteq \alpha \to \beta$. It is why $\lambda x.\, x\, x$ has no type. (Chapter 13)

Opposite category. The category $\mathcal{C}^{\mathrm{op}}$ with the same objects as $\mathcal{C}$ and every arrow reversed. By duality, every construction has a mirror image: terminal and initial objects, products and coproducts, monos and epis. (Chapter 16)

Outlives. The order on lifetimes: `'a: 'b`, read "`'a` outlives `'b`", means that a reference valid for `'a` is also valid for `'b`. The bound `T: 'a` says that every reference inside `T` outlives `'a`. (Chapter 26)

Ownership. Rust's rule that every value has an owner, the variable or place responsible for dropping it. When a value is moved, ownership moves with it, and so does the responsibility for dropping it. (Chapter 24)

E.16 P

Parameter, index. The two kinds of arguments of an inductive family. A parameter is the same in the result type of every constructor, like the `T` of Rust's `Option<T>`; an index may be chosen by each constructor, like the length in a type of vectors `Vec α n`. (Chapter 30)

Parametric polymorphism. One piece of code that works uniformly at every type and cannot tell which type it got, as in System F and in Rust's generics. (Chapter 12)

Parametricity. Reynolds's principle that polymorphic functions preserve relations between the types they are used at, from which the free theorems follow. It is a theorem about the language, not provable inside Lean, and Rust breaks it with features such as `Any` and `size_of`. (Chapter 18)

`partial`. A Lean marker for a definition that may not terminate. The function is compiled and runs, but for the logic it is opaque, and Lean accepts it only at a type that is inhabited, so it cannot prove a false proposition. (Chapter 2)

Permission. The reading of references as rights over a place: the owner may read, write and free; `&mut T` is a unique read-write permission and `&T` a duplicable read permission. (Chapter 26)

`PhantomData`. A zero-sized field that tells the Rust compiler to treat a struct as if it contained a value of some type. Typestate uses it to mention a state parameter, and it also chooses the variance of a type. (Chapter 25)

Π-type. The type $\Pi x : A.\, B$ of dependent functions, whose result type may depend on the argument, written `(x : A) → B` in Lean. The function type $A \to B$ is the case where B does not mention x, and under Curry–Howard Π is ∀. (Chapter 30)

Place, partial move. A place is an expression that denotes a location in memory: a local variable, possibly followed by projections such as a field `p.0` or a dereference. Rust tracks moves and borrows per place, so moving one field out of a struct, a partial move, leaves the other fields usable. (Chapter 22)

Polonius. A reformulation of Rust's borrow checking as Datalog rules over loans and origins (regions seen as sets of loans), which accepts safe programs such as "problem case #3" that NLL rejects. Bringing its reasoning into rustc is ongoing work: check the current status. (Chapter 29)

Preorder. A reflexive and transitive relation $\leq$. Every preorder is a thin category, one with at most one morphism between any two objects, and both subtyping and the outlives relation on lifetimes are preorders. (Chapter 16)

Preservation. The half of type safety which says that a well-typed term that takes a step is still well typed, with the same type. (Chapter 9)

Principal type. A type of a program of which every other type of the program is an instance, such as $(a \to a) \to a \to a$ for `twice`. In Hindley–Milner every typable program has one. (Chapter 13)

Product. An object with two projections through which every pair of morphisms into the components factors uniquely, a definition by universal property. In the category of types it is the pair type $A \times B$. (Chapter 16)

Progress. The half of type safety which says that a well-typed closed term is a value or can take a step: it is never stuck. (Chapter 9)

Promotion. The rule of linear logic that builds a duplicable value $!e$ only from a context in which everything is duplicable. In Rust it is the rule that a type can be `Copy` only if all its fields are (E0204). (Chapter 23)

Proof irrelevance. In Lean, any two proofs of the same proposition are definitionally equal, so a proof carries no information beyond the fact that it exists. (Chapter 10)

Proof term. The term that a proof is under Curry–Howard. Tactics build one, the kernel checks it, and `#print` shows it. (Chapter 4)

`Prop`. Lean's universe of propositions. Its elements are proved rather than computed, any two proofs of the same proposition are equal, and it is impredicative. (Chapter 10)

Proposition. Something that may be proved, such as $A \wedge B$: a piece of syntax that we talk about, as opposed to a judgment, which is an act of knowledge about it. Under Curry–Howard, a type. (Chapter 4)

E.17 R

RAII. "Resource acquisition is initialization": tie each resource to a value whose destructor releases it, so that the compiler releases the resource on every path out of the scope, including early returns and panics. (Chapter 24)

`Rc` and `Arc`. Reference-counted pointers that share a value among several owners instead of duplicating it: cloning one increments a counter, dropping one decrements it. `Arc` uses an atomic counter. Because of the counting, neither can be `Copy`. (Chapter 23)

Reborrow. A new reference made from an existing one for a shorter region, such as `&mut *r`, which the compiler inserts when a `&mut` is passed to a function. The original reference is usable again when the reborrow ends. (Chapter 28)

Recursor. The most general elimination principle of an inductive type, which Lean generates automatically for every `inductive` declaration. Folds and proofs by induction are uses of it. (Chapter 11)

Redex. A reducible expression: an application whose function part is an abstraction, $(\lambda x.\, t)\, s$. Under Curry–Howard, a detour. (Chapter 7)

Reduction strategy. A rule that chooses which redex to contract next. Normal order contracts the leftmost, outermost redex, also under λ, and finds a normal form whenever there is one; call-by-name and call-by-value never reduce under λ, and call-by-value, which is what Rust does, first reduces the argument to a value. (Chapter 7)

Refinement type. A type together with a predicate on its values, such as "integers between 0 and 100". Lean's subtypes are refinement types with proofs; Rust approximates them with newtypes that have private fields. (Chapter 30)

Region. A part of a program during which some memory may be used. Region calculi such as Tofte and Talpin's attach regions to pointer types; Rust's lifetimes are region variables, and under NLL a region is a set of program points. (Chapter 26)

Relevant logic. The substructural logic that keeps contraction and drops weakening: every assumption must be used at least once. (Chapter 21)

RustBelt. The project of Jung, Jourdan, Krebbers and Dreyer (POPL 2018) that proved, with proofs checked in Coq, the soundness of λRust, a core language close to MIR, and of standard library abstractions built with `unsafe`, by semantic typing in Iris, a higher-order concurrent separation logic, with a logic of lifetimes. Iris is affine: $P * Q \vdash P$. (Chapter 32)

E.18 S

Safety invariant, validity invariant. The two invariants of a type that `unsafe` code must respect. The validity invariant holds at all times, because the compiler relies on it (a `bool` is 0 or 1); the safety invariant holds whenever safe code can observe the value (a `String` holds UTF-8). (Chapter 32)

Semantic typing. Defining well-typedness by behavior instead of by derivations: $\Gamma \vDash e : \tau$ when e, run with good values for its variables, produces a good value of type τ. It covers safe abstractions built on `unsafe` code that no type checker accepts. (Chapter 32)

Separation logic. A logic for programs with pointers, due to Reynolds, O'Hearn and their collaborators, in which an assertion describes the piece of the heap that you own. Its separating conjunction $P * Q$ says that the heap splits into disjoint parts satisfying P and Q, and its frame rule makes proofs local. (Chapter 32)

Sequent. A judgment $\Gamma \vdash A$: from the assumptions in the context Γ, the formula A is true. (Chapter 4)

Sequent calculus. Gentzen's second presentation of logic, with right rules that prove a connective and left rules that use an assumption, instead of introductions and eliminations. It suits proof search and the study of the structural rules. (Chapter 4)

Session type. The type of a communication channel, describing the whole sequence of messages on it: $!\tau.\, S$ sends a τ, $?\tau.\, S$ receives one, *end* stops. The two ends of a channel have dual types, with sends and receives swapped. (Chapter 25)

Σ-type. The type $\Sigma x : A.\, B$ of dependent pairs, in which the type of the second component depends on the first. $A \times B$ is the case where B does not mention x, and under Curry–Howard Σ is ∃. (Chapter 30)

Simply typed λ-calculus. STLC. The λ-calculus with types: base types, functions, and in this book unit, booleans, naturals, pairs and sums. It is the smallest interesting type theory, every well-typed term terminates, and under Curry–Howard it is intuitionistic logic. (Chapter 8)

`sorry`. A Lean placeholder that proves anything. Lean accepts it with a warning and records it as the axiom `sorryAx`; no proof in this book depends on it. (Chapter 2)

Soundness, completeness. A system is sound for a semantics when everything it derives is true there, and complete when everything true there is derivable. The book proves a type checker sound for its rules (Chapter 3) and natural deduction sound for truth tables (Chapter 4); intuitionistic natural deduction is complete for Kripke models but not for truth tables (Chapter 5). Type safety is the soundness of a type system for its semantics (Chapter 9).

Stacked Borrows, Tree Borrows. Two proposed aliasing models for Rust, which say which uses of references and raw pointers are undefined behavior by tracking a stack, or a tree, of permissions for each location. Neither is, at the time of writing, the official definition; Miri can check programs against both. (Chapter 32)

Strict positivity. Lean's condition on inductive types: the type being defined must never appear to the left of an arrow in the arguments of its constructors. Without it, non-terminating terms could be built, and with them a proof of `False`. (Chapter 11)

Structural recursion, well-founded recursion. Lean's two ways to see that a recursive function terminates. In structural recursion every call is on a piece of the argument, and Lean accepts it automatically; in well-founded recursion a measure decreases at every call, named with `termination_by` and proved with `decreasing_by` when Lean cannot find the proof itself. (Chapter 2)

Structural rules. The rules that mention no connective and talk only about the context: exchange, weakening and contraction. Which ones a logic keeps decides what an assumption is. (Chapter 6)

Stuck. A term that is not a value and cannot take a step, such as `true + 5`: the formal version of a crash. Type safety says that well-typed programs never get stuck. (Chapter 1)

Substitution. Replacing the free occurrences of a variable by a term, written $t[x := s]$ or $t[s/x]$. It must be capture-avoiding: bound variables are renamed when necessary, so that no free variable of the substituted term is captured. In type inference, a substitution is a map from type variables to types. (Chapter 7)

Substitution lemma. Substituting a term of the right type for a variable preserves typing. The proof of preservation depends on it, and in a cartesian closed category it says that substitution is composition. (Chapter 9)

Substructural logic. A logic obtained by deleting some of the structural rules: affine, relevant and linear logic, and ordered logic, which drops exchange as well, so that assumptions must be used in order. Which rules are deleted decides what an assumption is: a fact, a resource that may be discarded, an obligation, or a resource that must be used exactly once. (Chapter 6)

Subtype. The book uses the word in two senses, and Chapter 27 warns against confusing them. In Lean, a subtype `{x // p x}` is the type of values paired with a proof of a property, a refinement type (Chapter 11). In subtyping, σ is a subtype of τ, written $\sigma <: \tau$, when a σ can be used wherever a τ is expected (Chapter 27).

Subtyping. A preorder $\sigma <: \tau$ on types, used through the rule of subsumption: a term of type σ may be used at type τ. Rust's only subtyping is between lifetimes, as when a `&'static T` is used as a `&'a T`; Lean has none. (Chapter 27)

System F. The polymorphic λ-calculus of Girard and Reynolds, with type abstraction $\Lambda\alpha.\, e$, type application and types $\forall\alpha.\, \tau$. It is impredicative; Rust's generics behave like its rank-1 fragment, with trait bounds added. (Chapter 12)

E.19 T

Tactic. An instruction that transforms the current goal of a Lean proof, closing it or replacing it with simpler goals. A proof written with tactics, in tactic mode, builds a proof term. (Chapter 1)

Terminal object, initial object. An object is terminal when every object has exactly one morphism to it, and initial when it has exactly one morphism to every object. In the category of types they are the unit type and the empty type. (Chapter 16)

Totality. The property of a function that returns a value for every input, and in particular terminates. Lean requires it of every definition that proofs can use, because a single non-terminating definition could prove `False`. (Chapter 2)

Trait object. A value of type `dyn Trait`, reached through a pointer that carries the data pointer together with a pointer to a vtable, the table of the trait's methods for the hidden type. It is an existential package, and the vtable is the run-time form of a dictionary. (Chapter 15)

Transport. Using a proof of `a = b` to turn a statement or a value about `a` into one about `b`; in Lean, `h ▸ e`, `subst` and `rw`. It is the elimination rule of the equality type. (Chapter 30)

Two-phase borrows. The rule that a mutable borrow inserted by the compiler, as in `v.push(v.len())`, is first only reserved, behaving like a shared borrow, and becomes active at its first use. (Chapter 29)

Type class. A named set of operations with their types, together with instances that declare which types belong to it; Wadler and Blott's design for principled ad-hoc polymorphism. Rust's traits and Lean's classes are type classes, and a Lean class can also require proofs of laws. (Chapter 14)

Type constructor. Something like `Option` or `Vec`, of kind `Type → Type`: a function from types to types, not a type itself. (Chapter 17)

Type inference. Reconstructing the types a programmer left out, by treating them as unknowns and solving the equations that the program imposes on them. Rust's inference is local to function bodies, with signatures as boundaries. (Chapter 13)

Type safety. The theorem that well-typed programs do not go wrong: by progress and preservation, they never reach a stuck state. For safe Rust, going wrong means undefined behavior, not panics, leaks or non-termination. (Chapter 1)

Type system. A set of rules that decides, before a program runs, which programs are well typed. The mathematical study of type systems is type theory. (Chapter 1)

Typestate. A protocol encoded in types: each state is a type, and each operation consumes a value of the type of the state where it is allowed and returns a value of the type of the next state. It works in Rust because values are affine. (Chapter 25)

E.20 U

Undefined behavior. The kind of going wrong that safe Rust rules out, such as reading freed memory, data races and using uninitialized values, to which the language assigns no meaning at all. (Chapter 1)

Unification. Solving an equation between types, $\tau_1 \doteq \tau_2$, by finding a substitution, a unifier, that makes the two sides equal. Inference wants the most general unifier, through which every other unifier factors. (Chapter 13)

Unit type. The type with exactly one value, `()` in Rust and `Unit` in Lean, written $\mathbf{1}$. It is "true" under Curry–Howard and a terminal object in the category of types. (Chapter 1)

Universal property. A definition of an object by how morphisms relate to it, such as "an object with two projections through which every pair of morphisms factors uniquely". It determines the object up to unique isomorphism. (Chapter 16)

Universe. A type of types. Lean has `Prop` for propositions and a hierarchy `Type 0`, `Type 1`, ... for data, each a member of the next; internally they are all written `Sort u`. (Chapters 10 and 12)

`unsafe`. A Rust keyword that marks a proof obligation the compiler cannot check: an `unsafe` block says that the code inside discharges the obligations of its operations, and an `unsafe fn` passes its obligations to the caller. (Chapter 32)

Usage theorem. The theorem of the affine λ-calculus that in a well-typed term every variable of the context occurs at most once. Chapter 23 generalizes it: variables whose type is not `Copy` occur at most once. (Chapter 22)

E.21 V

Variance. How a subtyping between the arguments of a type constructor carries over to the constructed types: covariant, contravariant, invariant or bivariant. In Rust it matters for lifetimes, and getting it wrong lets safe code produce a dangling reference. (Chapter 27)

E.22 W

Weakening. The structural rule that lets an assumption go unused. In programs it is discarding a value, which in Rust runs the value's destructor: weakening with a side effect. (Chapter 6)

E.23 Y

Yoneda lemma. The theorem that the natural transformations from $\mathrm{Hom}(\alpha, -)$ to a functor F correspond to the elements of $F\,\alpha$. For types it reads $\forall \beta.\, (\alpha \to \beta) \to F\,\beta \cong F\,\alpha$, which is continuation-passing style. (Chapter 18)

E.24 Z

Zero-sized type. A type whose values need no bytes at all, such as `()`, the unique type of a `fn` item or `PhantomData`. (Chapter 11)